

Transfer-ease claim (C2 evidence)

Mission: hardware bag `drone1_20260812_030151` vs Isaac velocity-ZOH replay. Plant: `starling2max.usd`. Date of this inventory: 2026-09-01.

This note is **only** the “transfer was easy” claim. It is measured by diffs at the simulation / hardware boundary (files, launch args, params, and sim-vs-hardware conditionals). Tracking-error numbers belong in `REPORT.md` and must not be reused here.

Two claims that must stay separate

Claim	What counts as evidence	What does not
Transfer was easy	How many launch/param/code files change when the same mission moves from hardware to Isaac, and whether those changes sit at the interface	Overlay RMSE, step-response τ , path length
The sim predicts hardware	Tracking-error agreement (pose overlay, dead time, τ)	A short file-diff list

A large fidelity gap can coexist with easy transfer: the autonomy graph above the interface can be unchanged while the closed-loop plant still does not match. That is the situation for this mission. XY RMSE 2.61 m and Z RMSE 3.43 m in `REPORT.md` say nothing about whether the workflow was a small swap. Suggested paper split: **Sec. VI-A** = this inventory; **Sec. VI-B** = fidelity.

Scope of “the same mission”

Hardware did **not** fly AirStack DROAN / `trajectory_controller`. The bag’s primary input is `v_des` on `/drone_1/fmu/velocity_command` (ZOH 20 Hz) from `LocalWaypointController` (`mission.json`). AirStack pieces that *are* in the bag: `odometry_conversion` and the `/fmu` uXRCE-DDS surface.

Isaac did **not** re-plan those waypoints. It replayed the recorded `TwistStamped` stream into PX4 SITL as `v_des`. Layers above the interface were therefore **not** on the control path on either side. That is the right boundary for a transfer-ease claim about *this* mission. It is the wrong boundary for a claim that “the whole AirStack autonomy graph transferred.”

Hardware launch files from the flight day are **not** in this repository. Hardware side is inferred from bag topics plus the repo’s documented hardware path (`px4_interface.launch.xml`, VOXL compose `sim:=false`).

A. Configuration and parameter file diff

Classification:

- **(a) backend selection** — which plant, plugin, transport, or container is started

- **(b) tuned value** — a number that could have been copied or retuned
- **(c) code path change** — a different branch executes
- **(d) topic / frame change** — namespace, frame id, or ENU/NED

“Δ lines” is the size of the *difference that was actually exercised* for this mission (file length when the files are alternatives; changed lines when it is the same file with different args).

Files that differ

File	Isaac (this run)	Hardware (inferred)	Class	Δ lines
interface/interface_bringup/launch/interface.launch.py	Used. MAVROS + <code>mavros_interface::MAVRosInterface</code> under <code>/robot/interface.FCU URL udp://:{14540+id}@{SIM_IP}:{14580+id}</code> .	Not used.	(a)	142 (entire file present only in sim)
interface/px4_interface/launch/px4_interface.launch.xml	Not used.	Used. Plugin <code>px4_interface::PX4Interface</code> under <code>/robot/fmu</code> . Bag topics <code>/drone_1/fmu/velocity_command</code> , <code>/drone_1/fmu/out/vehicle_odometry</code> .	(a)	82 (entire file present only on hardware)
autonomy_bringup/launch/robot.launch.xml	<code>sim default true</code> → <code><set_parameter name="use_sim_time" value="true"/></code> . <code>role:=full</code> . <code>urdf_file default iris_with_sensors.pegasus.robot.urdf</code> .	VOXL/L4T compose passes <code>sim:=false</code> . <code>role</code> would be <code>onboard</code> if AirStack autolaunch were used. This flight's high-level controller is outside AirStack.	(c) clock; (a) URDF unused by PX4 plant	1 line (<code>use_sim_time</code>); 3 group if <code>role</code> blocks (lines 36–78) are role dispatch, not sim-vs-hw
desktop_bringup/launch/robot.launch.xml	Wrapper used (<code>LAUNCH_PACKAGE=desktop_bringup</code>). Passes <code>sim</code> through; optional RViz.	Not used (<code>LAUNCH_PACKAGE=autonomy_bringup</code> on VOXL).	(a)	21
robot/docker/docker-compose.yaml robot-desktop vs robot-voxl-onboard	<code>LAUNCH_PACKAGE=desktop_bringup</code> , <code>AUTONOMY_ROLE=full</code> , <code>ROBOT_NAME_SOURCE=container_name</code> , launch without <code>sim:=false</code> .	<code>LAUNCH_PACKAGE=autonomy_bringup</code> , <code>AUTONOMY_ROLE=onboard</code> , <code>ROBOT_NAME_SOURCE=hostname</code> , <code>REAL_ROBOT=true</code> (image build), <code>sim:=false</code> .	(a)+(c)	~12 environment / command lines
simulation/isaac-sim/launch_scripts/starling2max_velocity_replay.py	Isaac plant + PX4 SITL. Spawn XY = hardware start; Z raised 0.08 m so the USD mesh is not in the Grid ground. Mass 0.557 kg.	N/A (physical Starling 2 Max).	(a); spawn Z is (b) geometry bias	232 (sim-only); spawn Z is 1 literal
mavros_interface/scripts/replay_velocity_zoh.py	Test injector. Publishes <code>/robot/interface/velocity_command</code> .	N/A. Hardware controller was live.	(c) test harness only — not a stack branch	340
tests/hw_sim_compare/{extract,run,plot}_velocity_replay.py	Host-side extract / run / overlay.	N/A.	(c) test harness	56 + 135 + 173

File	Isaac (this run)	Hardware (inferred)	Class	Δ lines
interface/mavros_interface/src/mavros_interface.cpp velocity_callback	FRAME_LOCAL_NED, ENU xyz as-is, IGNORE_YAW_RATE, yaw=0. MAVROS converts ENU→NED.	Hardware plugin converts ENU→NED itself (next row).	(d)	~25 lines of this callback
interface/px4_interface/src/px4_interface.cpp velocity_callback	Not used.	vN=v _y , vE=v _x , vD=-v _z ; yaw-rate negated. Direct TrajectorySetpoint.	(d)	23
PX4 SITL params set live	MPC_XY_VEL_P_ACC=1.8, MPC_Z_VEL_P_ACC=8.0, MPC_XY_VEL_MAX=0.6, MPC_XY_P=0.95, MPC_Z_P=5.0	Bag has no ulog. Values taken from mission.json / Neelay SE3 defaults (01_21_03_position_mode.ulg), assumed not verified on this flight. Indoor D0012 params were not loaded.	(b)	5 values
Robot namespace	ROBOT_NAME=robot_1 (compose replica)	drone_1 (bag)	(d)	prefix only
local_bringup/launch/local.launch.xml	local_interface_ns default /{ROBOT_NAME}/interface; local_extended_state_topic = {ns}/mavros/extended_state.	Documented hardware override: local_interface_ns:={ROBOT_NAME}/fmu. This mission did not close the loop through local bringup.	(d)	2 launch args (lines 17–21). File is 223 lines; defaults were not swapped by sim:=false
perception_bringup/launch/perception.launch.xml	launch_natnet default false. Stereo disparity still launched (unused by velocity replay).	Bag ground truth is /drone_1/pose (mocap). NatNet is a separate LAUNCH_NATNET toggle, not a sim if-block.	(a) sensor source	launch_natnet group ~4 lines; no sim-vs-hw if
macvo_ros2/config/zedcam_config.yaml	use_sim_time: false with a comment to flip in sim. Global use_sim_time from robot.launch.xml overrides when sim:=true.	false.	(c) clock	1
interface/robot_interface/src/robot_interface_node.cpp	createSharedInstance("mavros_interface::MAVROSInterface") is hardcoded; the interface ROS param is read and then ignored.	px4_interface.launch.xml sets interface:=px4_interface::PX4Interface, which this node would not honor if the same binary were used. Hardware launch starts the same exec with the param, so this is a latent bug on both sides.	(c)	1 line (line 131)

Files that do not differ for this mission

These are the same source on both sides and were not retuned for the replay:

- All of **behavior**, **global planning**, **DROAN**,

trajectory_controller, **pid_controller** — not on the v_{des} path.

- sensors.launch.xml — no sim if.
- No #ifdef SIM / #ifdef HARDWARE in AirStack interface, local,

perception (AirStack-authored), or behavior packages.

Unified diff of the two *alternative* interface launch files (interface.launch.py vs px4_interface.launch.xml) is 141 added/removed lines. That number overstates the operational change: the files are different backends, not a patch of one file. Unified diff of the two plugin .cpp files is 680 lines for the same

reason (two implementations of `RobotInterface` , not a 680-line edit to fly the mission).

What actually had to be touched to run this replay

Operational delta, excluding test harness scripts that do not ship on the vehicle:

1. **Backend (a):** start Isaac + PX4 SITL + MAVROS instead of uXRCE-DDS

+ `px4_interface` (~142 vs 82 launch lines; different transport).

1. **Topic (d):** publish on `/robot/interface/velocity_command`

instead of `/robot/fmu/velocity_command` .

1. **Frame (d):** MAVROS `FRAME_LOCAL_NED` with ENU as-is. Hardware

plugin does the ENU→NED conversion in-process. Same `v_des` numbers; different adapter.

1. **Clock (c):** `sim:=true` → `use_sim_time` . One line.

2. **Tuned values (b):** five PX4 MPC gains copied from documented

hardware defaults; spawn Z +0.08 m so the USD is not in the ground plane.

1. **Not done:** no planner gains, no perception params, no BT, no

indoor PX4 param set.

The test-only injector (`replay_velocity_zoh.py` , 340 lines) is how hardware `v_des` was fed into SITL. It is not a sim-vs-hardware branch inside the autonomy stack.

B. Sim vs hardware branching, by layer

Question: does any conditional branch on simulator vs hardware **above the interface layer**?

Interface layer (expected)

Location	What branches	Kind
<code>interface.launch.py</code>	<code>if sim_type != 'simple':</code> skip MAVROS. <code>SIM_TYPE</code> is unset on this Isaac run (MAVROS starts). <code>simple-robot</code> compose profile sets <code>SIM_TYPE=simple</code> .	env <code>SIM_TYPE</code>
<code>interface.launch.py</code>	Plugin string hardcoded to <code>maavros_interface::MAVROSInterface</code> . Hardware uses a different launch file , not an if-block in this one.	backend selection by file swap
<code>robot_interface_node.cpp:131</code>	<code>createSharedInstance</code> ignores the <code>interface</code> param and always loads MAVROS.	latent; not a sim <code>if</code>
<code>maavros_interface.cpp velocity_callback</code>	Frame/type mask for MAVROS <code>PositionTarget</code> . Not <code>if (sim)</code> .	transport adapter

Location	What branches	Kind
px4_interface.cpp velocity_callback	ENU→NED in the plugin. Not <code>if (sim)</code> .	transport adapter
#ifdef	None in <code>interface/</code> (AirStack-authored).	—

Autonomy bringup (above interface)

Location	What branches	Sim vs hardware?
robot.launch.xml:18	<code><set_parameter name="use_sim_time" value="true" if="\$(var sim)" /></code>	Yes — clock only. Default <code>sim</code> is <code>true</code> . Hardware compose passes <code>sim:=false</code> .
robot.launch.xml:36–78	<code>role ∈ {full, onboard, offboard}</code>	No . This is compute placement (desktop vs VOXL vs GCS), not plant. Desktop Isaac used <code>full</code> ; VOXL autolaunch would use <code>onboard</code> .
onboard_autonomy_all.launch.xml	Empty-string skip for a layer package	Not sim-vs-hw.
onboard_autonomy_local.launch.xml	Clears <code>global_package</code> / <code>logging_package</code>	Role, not sim.

Yes, there is branching above the interface: global `use_sim_time` when `sim:=true`. It does not change planners, controllers, or topics.

`sim:=false` does **not** swap MAVROS for `px4_interface`. The default graph always includes `interface.launch.py`. Hardware uXRCE-DDS is a **separate launch file** that must be started instead of (or in addition to) that default. That gap is itself transfer evidence: the documented hardware path is a file swap at the interface, not a flag that rewires local/perception/behavior.

Local layer (above interface)

Location	What branches	In this mission?
local.launch.xml	No <code>if sim</code> . Comments document <code>local_interface_ns</code> <code>/interface</code> vs <code>/fmu</code> . Default is <code>/interface</code> .	Not exercised (replay injects at the interface).
attitude_controller.cpp	Param <code>thrust_sim</code> (default <code>true</code>). <code>if (thrust_sim)</code> uses <code>sqrt(thrust/g)*hover_throttle</code> ; else a battery-voltage quadratic. Also skips battery hover-throttle when <code>thrust_sim</code> . Same pattern in <code>controller_ekf.cpp</code> (<code>thrust_sim</code> default <code>false</code> there).	Not in the loop . Hardware flew offboard velocity, not this attitude controller.
trajectory_controller.cpp	Log string contains “sim time”; not a sim/hw branch.	No.
DROAN / disparity expansion	No <code>sim if</code> .	No.

Yes, there is a real sim-vs-hardware code path above the interface: `thrust_sim` in `attitude_controller`. It is unused for this velocity replay. It *would* matter for a roll-pitch-yawrate-thrust transfer of the same stack.

Perception / sensors (above interface)

Location	What branches	In this mission?
perception.launch.xml	launch_macvo, launch_stereo_image_proc, launch_natnet – feature toggles, not if sim.	Velocity replay does not consume them. Hardware pose is /drone_1/pose.
natnet_ros2.launch.py / vision_pose_converter.launch.xml	use_sim_time launch arg (default false).	Clock only.
zedcam_config.yaml	Comment to set use_sim_time in sim.	Clock only.
sensors.launch.xml	None.	–

No algorithm-level sim vs hardware if in perception or sensors for this mission.

Behavior / global

No sim-vs-hardware conditionals in behavior/. The only sim if in global/ is exploration/launch/robot_launch_gazebo.xml (Gazebo legacy use_sim_time), not used here.

Docker / image (below or beside the stack)

Location	What branches
Dockerfile.robot REAL_ROBOT	Skips Foxglove Studio install on real-robot / non-amd64 images. Not an autonomy branch.
Compose SIM_TYPE=simple	Only the simple-robot profile. Not this Isaac run.

Direct answer

Branching above the interface exists, but it is thin:

1. **Clock:** use_sim_time gated by launch arg sim (bringup).
2. **Thrust mapping:** thrust_sim in attitude_controller (local / control). Present in the codebase; **not used** on this mission's command path.

1. **Interface namespace default:** local bringup still points at /interface unless an operator overrides it. Not an automatic if (sim) – a documented remap.

There is **no** planner, world-model, perception algorithm, or behavior tree if (sim) that was required to replay this bag. #ifdef sim/hw split: **none** in AirStack-authored autonomy code.

Verdict on “transfer was easy”

For this mission, transferring v_des from the Starling bag into Isaac PX4 SITL is a **small, interface-local swap**:

- Two alternative interface launch files (142 vs 82 lines), not a

rewrite of local/perception/behavior.

- One topic rename (`/fmu/velocity_command` → `/interface/velocity_command`).
- One frame-adapter difference (who does ENU→NED).
- One clock flag.
- Five PX4 velocity-loop numbers copied from documented hardware

defaults; one spawn-Z bias for the USD.

That is the C2 evidence. It supports “the command path did not require retuning the layers above the interface.”

It does **not** support “the sim predicts hardware.” The overlay in ``REPORT.md`` shows a large gap (sim τ $\sim 3.7\times$ faster; XYZ RMSE 4.31 m; sim walks a 27 m scribble vs hardware 7.7 m polyline). Those numbers belong in the fidelity section. Using them here would confuse a short wiring list with a physics match.